

Current Solutions and Problems in MongoDB Distributed Systems using docker

1. Introduction

From an architectural perspective, MongoDB's distributed model is built around two main mechanisms. Replica sets provide redundancy and automatic failover by maintaining the same dataset across multiple nodes, typically one primary and multiple secondaries; MongoDB's own documentation recommends three data-bearing members as the minimum production-style configuration. When scale-out for data volume or throughput becomes necessary, MongoDB adds sharding, where data are partitioned across shards through a shard key, with config servers storing cluster metadata and mongos routers dispatching requests. On the container side, Docker Compose defines services, networks, and volumes in a single YAML file and, by default, creates one application network where services can reach each other by name.

2. Existing Solutions

Makris et al.

Makris et al. study MongoDB versus PostgreSQL for large spatio-temporal workloads using a five-node cluster on Amazon Web Services EC2 and an 11 GB vessel-tracking dataset provided by MarineTraffic. Their objective is not to prove that one database is universally better, but to test which model is more efficient for realistic industrial query patterns over large spatial and temporal data. Their methodology uses real-world query scenarios, repeated executions, and indexed versus non-indexed runs. The main finding is that PostgreSQL outperforms MongoDB in most queries, with an average speedup of about 2.1, but MongoDB performs better for polygon intersection queries. They also show that indexing radically changes results, especially for MongoDB, and that the absence of indexes can make execution orders of magnitude slower.

Gkamas et al.

Gkamas et al. evaluate distributed database strategies for industrial IoT data using Docker-based deployments. Their testbed compares MongoDB against two PostgreSQL variants, one relational and one JSON-based, across standalone mode and distributed clustered deployments with two, four, six, and eight nodes. MongoDB is deployed with a router, configuration server, and sharded servers, while nodes communicate through OpenVPN-secured Docker containers. The methodology focuses on three benchmark scenarios that are central to practical applications: inserts, select/find queries, and correlation/aggregation queries. The main result is nuanced: PostgreSQL with JSON has better insert latency in some smaller-scale setups, but MongoDB often provides much higher throughput; with six or eight shards, MongoDB reaches roughly twice PostgreSQL's throughput on insert workloads and also performs better on distributed selection workloads. Their paper is relevant to my project as it explicitly studies containerized, horizontally scaled database deployments rather than only single-node benchmarks.

Mehmood et al.

Mehmood et al. address a different but equally important problem: how to model temporal sensor data correctly in MongoDB. Their objective is to design a logical, compact, and evolvable schema for real-time ANT+ sensor streams. The paper proposes a middleware-based schema model, implemented with Node.js and Mongoose, that stores hierarchical JSON documents and evolves them during ingestion without sacrificing schema flexibility. The prototype architecture uses a WebSockets-based real-time interface and a Node.js data server that accepts, processes, and stores JSON messages in MongoDB. Rather than emphasizing benchmark charts, the authors focus on temporal data modeling choices, denormalization, and algorithmic schema evolution. Their main contribution is conceptual and architectural: they show how MongoDB can support real-time sensor workloads when the document model is designed around time granularity and embedded hierarchy instead of relational-style normalization.

Jurado Pérez and Salvachúa

Jurado Pérez and Salvachúa propose a cloud-based e-health IoT reactive multi-service architecture for elderly care in smart homes. MongoDB serves as the distributed storage backend, deployed as a cluster on Kubernetes, while other components include EMQ, Kafka, Spark, connectors, mobile apps, and cloud-native deployment pipelines. The authors' methodology is stronger than a simple architectural sketch because they validate the system with multiple workload experiments using Apache JMeter on Kubernetes clusters. They benchmark MongoDB-backed microservices for CRUD-style operations, messaging throughput for the broker layer, and emergency alert response time under increasing load. The system's functionality is broad: it supports health data ingestion, diet-management services, reactive alerting, and near-real-time analytics. Quantitatively, horizontal scaling from one to two pods improved the "Save" microservice throughput from 254.43 to 341.05 requests/s, a 34% increase, while the Kafka cluster sustained 18,000 publishing requests/s with an average response time of 669 ms in their acceptable operating point.

Ferreira et al.

Ferreira et al. focus on a problem that is directly relevant to distributed MongoDB deployments: the cost of stronger consistency in cloud-based, geo-replicated NoSQL systems. Their paper compares MongoDB, Cassandra, and Redis in both single-region and multi-region cloud deployments and explicitly varies consistency strength. For MongoDB, they configure a three-node cluster using the official Docker image, compare `readConcern: local` against `linearizable`, and compare `writeConcern: 1` against `writeConcern: 3`. The experiments are run with Locust under multiple concurrency levels, and the authors analyze throughput, response time, and resource usage. Their conclusions are highly important for benchmark design: MongoDB is relatively strong in read response time, but writing performance is heavily affected by stronger consistency, especially in the multi-region case. In multi-region reading, MongoDB loses roughly 57% of throughput and sees about a 237% response-time increase when moving from weak to strong consistency.

Pacella et al.

Pacella et al. present a recent nine-layer framework for cloud manufacturing that combines Kafka for ingestion, Spark Streaming for real-time analytics, MongoDB for data management, and Docker plus Kubernetes for microservice deployment. Their methodology is comparative and experimentally grounded: they test Kafka versus RabbitMQ for ingestion, Spark Streaming versus Hadoop for processing, and microservices versus a monolithic Flask application for the application layer. MongoDB is part of the persistence backbone rather than the only variable under study, but that actually makes the paper highly relevant to my project because it shows how MongoDB behaves inside a more realistic distributed pipeline. Their results show that Kafka achieves 714.21 messages/s versus 671.55 for RabbitMQ under a 1000-message workload, Spark Streaming processes a representative workload in 4.236 s versus 5.976 s for Hadoop, and the microservice architecture dramatically outperforms the monolith under heavy load. At 10,000 users, the ingestion microservice processed 366,940 requests while the monolithic version handled 102,972, and the microservices kept sub-10 ms response times while the monolith exceeded 9000 ms. The paper also provides public dataset access through the UCI repository.

3. Discussion

Makris et al.

The main strength of Makris et al. is realism. They use a genuine industrial spatio-temporal dataset, a multi-node cluster, and complex queries that are much closer to applied analytics than toy CRUD workloads. For my project, this paper is valuable because it proves that MongoDB's document model is not automatically the best choice once the workload becomes analytically complex, spatially rich, and heavily index-dependent. Its main weakness is also its specificity: the study is dominated by spatial and temporal analytical queries, not by the write-heavy operational loads that are typical in many replica-set-backed applications. As a result, it is excellent for demonstrating query-planning and indexing risks, but less useful for predicting the behavior of a simpler Dockerized CRUD service. The paper states that the dataset was provided for research by MarineTraffic, which means reproducibility is constrained compared with papers that release raw benchmark data directly.

Gkamas et al.

The strength of Gkamas et al. is direct relevance to containerized scaling. Among the papers I reviewed, this is the one that most closely resembles the evolution path of my own project: start from a simple service, containerize the nodes, then scale horizontally and watch how response time, throughput, and failure rates behave. The results are especially useful because they show that MongoDB's advantage is workload-sensitive. It is not always the latency winner, but it becomes attractive when throughput and scale-out matter more, especially under larger sharded deployments. The weakness is that MongoDB is benchmarked mostly against competing systems rather than tuned internally across a wide range of MongoDB-specific parameters such as read concerns, write concerns, or index strategies. Another practical limitation is reproducibility: the paper's accessible text explicitly marks the data availability statement as "not applicable."

Mehmood et al.

Mehmood et al. contribute something many performance papers neglect: data modeling discipline. Their strongest insight is that MongoDB's benefits depend on choosing an appropriate document shape. A poorly normalized or poorly nested schema can erase much of the advantage of a document database, especially for time-series or real-time sensor workloads. This is highly relevant to my own benchmark planning because it suggests that I should not benchmark "MongoDB" in the abstract; I should benchmark a specific schema design. The main weakness is the lack of a strong quantitative evaluation. The paper convincingly explains why its model should scale and remain flexible, but it gives far less benchmark evidence than the later cloud-native papers. That weakness actually becomes a lesson for my project: if I adopt schema design claims from this paper, I should verify them with my own measurements rather than accept them as self-evident. On the reproducibility side, the paper is better than many older design papers because it references the NodeTempANT prototype on GitHub.

Jurado Pérez and Salvachúa

This paper's major strength is end-to-end system thinking. Unlike narrow database comparisons, it shows MongoDB operating inside an actual reactive cloud architecture with messaging, stream processing, mobile clients, Kubernetes deployment, and CI/CD through GitLab. For my project, that matters because a database benchmark on its own can miss bottlenecks introduced by queues, APIs, or orchestration layers. The experiments also show meaningful scaling behavior, especially the 34% throughput improvement for the write-oriented "Save" service after horizontal scaling. The weakness is that the benchmark environment relies heavily on artificial data and a domain-specific architecture, so the exact numbers should not be generalized too broadly. Another issue is that some of the most severe bottlenecks in the paper come from the surrounding infrastructure, such as the custom MQTT-Kafka bridge, meaning not every performance limit observed there is truly a MongoDB limit. Reproducibility is relatively strong: the paper provides supplementary spreadsheets for microservice, EMQ, Kafka, and emergency-service experiments.

Ferreira et al.

Ferreira et al. provide the clearest evidence for one of the central trade-offs in distributed systems engineering: stronger consistency costs performance, and the cost grows when replicas are geographically separated. This is a direct warning for my next phase. If I only benchmark local, weakly consistent reads and writes, I will miss a major part of MongoDB's real distributed behavior. The paper is methodologically strong because it uses controlled cloud deployments, identical load conditions, explicit consistency settings, and statistical significance testing. Its weakness is scope: it studies a three-node cluster under cloud VMs rather than a more complex sharded deployment, and it is designed as a cross-system comparison rather than a deep MongoDB tuning study. Still, for benchmark planning, it is probably the single most important article I reviewed. It is also the strongest paper on artifact release, because the authors publish raw CSV results through a public Google Drive.

Pacella et al.

Pacella et al. are useful because they reflect the current direction of distributed-platform research: MongoDB is increasingly studied as one layer in a broader cloud-native pipeline rather than as an isolated database server. The paper shows that once

ingestion, analytics, and API layers are containerized and independently scaled, microservice architectures can heavily outperform monolithic ones under stress. For my project, that is a meaningful signal that the database benchmark should eventually be embedded in a service benchmark, not left as a database-only experiment. The weakness is that MongoDB is not isolated as the single experimental variable, so the paper is less precise about MongoDB-specific knobs than Ferreira or Gkamas. Also, although the framework is modern, the experiments were run on a comparatively modest single machine, which limits how far its absolute numbers can be generalized. Reproducibility is decent because the authors publish the UCI dataset source and include detailed appendix pseudocode, but they do not expose a full public code repository in the accessible article text.

4. Cross-paper comparison

Across the six papers, the most consistent pattern is that MongoDB performs best when the workload is semi-structured, write-heavy, ingestion-oriented, or naturally aligned with denormalized JSON documents. It becomes less convincing when the workload is dominated by complex relational analytics, certain spatial queries, or stronger consistency guarantees across distant replicas. Another repeated lesson is that scale-out is not free: sharding improves throughput and capacity, but it also increases architectural complexity and makes data modeling decisions, especially shard-key and document-shape decisions, far more important. The literature therefore does not support simplistic claims such as “MongoDB is faster” or “SQL is faster”; instead, it shows that workload shape, indexing, topology, and consistency settings determine the outcome.

A second pattern is the shift from simple database comparisons to full cloud-native stacks. Earlier work such as Mehmood et al. centered on schema design, while later work such as Jurado Pérez, Ferreira, and Pacella increasingly evaluates MongoDB inside Kubernetes, cloud regions, streaming pipelines, and microservice ecosystems. This shift matters for my project because it suggests that a benchmark limited to basic CRUD on a local replica set would be too narrow for a modern distributed-systems evaluation. At minimum, I should test replication, failover, consistency, indexing, and service-level behavior under concurrent load.

Based on the papers and the official documentation, the key strengths of MongoDB as a distributed platform are clear: a document-native model that fits heterogeneous sensor and application data well, built-in replication and failover, native sharding for very large datasets and high throughput, and a strong ecosystem for containerized deployment and integration with event-driven systems. Its key limitations are equally clear: sensitivity to index and schema design, performance penalties under strong consistency or cross-region coordination, higher cost for distributed transactions and multi-shard work, and operational complexity once the system moves from a replica set to a true sharded cluster. MongoDB’s own documentation reinforces this interpretation by noting that sharding is intended for very large datasets and high-throughput operations, that linearizable reads can be significantly slower than weaker read concerns, and that distributed transactions cost more than well-designed single-document operations.

On reproducibility, the strongest open-resource support comes from Ferreira et al., who publish raw CSV results; Jurado Pérez and Salvachúa, who publish supplementary benchmark spreadsheets; Mehmood et al., who reference a prototype repository; and Pacella et al., who publish the dataset source they used. By contrast, Makris et al. rely on a research-provided dataset from MarineTraffic, and Gkamas et al.

explicitly state that data availability is not applicable. This matters for my report because it shows that not all “peer-reviewed evidence” is equally reusable for my own benchmark design.

5. Implications for My Project

The literature changes how I should approach the next stage of my project. Because my current system is a three-node Docker-based replica set, the next logical benchmark target is not an immediate jump to a large sharded deployment. Instead, I should first benchmark the replica-set topology I already have, because the sources show that replication strategy, consistency level, indexing, and failover behavior already produce large differences in throughput and latency. Only after I understand those effects should I extend the platform toward sharding, which MongoDB documentation treats as a higher-complexity scale-out mechanism involving config servers, routers, and per-shard replica sets.

Based on the evidence, the benchmarks I design next should cover five dimensions. First, I should measure failover and recovery by forcing a primary-container stop and timing reelection, write recovery, and read availability. Second, I should compare weak and strong consistency settings, because Ferreira et al. show that these choices can substantially change both throughput and response time, especially beyond a single local region. Third, I should test indexed versus non-indexed queries, because Makris et al. show that indexing can dominate the outcome. Fourth, I should separate workload types into inserts, point reads, list reads, and aggregations, because Gkamas et al. and Jurado Pérez and Salvachúa both show that different operations scale differently. Fifth, if time permits, I should compare the current replica set with a small sharded deployment to observe whether the added operational complexity actually produces a throughput gain in my workload.

For tooling, a sensible plan would be to combine database-level and service-level benchmarking. The YCSB paper remains a standard reference for cloud-serving database benchmarking and is explicitly designed to compare modern distributed data-serving systems. At the same time, the current papers I reviewed frequently use HTTP-level tools such as Locust or Apache JMeter to stress APIs and distributed services. Therefore, the next stage of my project should probably include one database-centered workload generator and one service-centered load generator, so that I can distinguish raw MongoDB behavior from the behavior of the full Dockerized application stack.

In practical terms, my next steps are now clearer. I should first formalize the workloads and metrics: throughput, response time, error rate, failover time, replication lag, and resource usage. I should then run repeatable experiments on the existing three-container replica set, varying indexing and consistency settings. After that, I should decide whether the evidence justifies a second benchmark phase with sharding or with a microservice layer around MongoDB. In other words, the literature suggests that benchmark design should progress from replica-set correctness, to performance under load, to scale-out complexity, rather than jumping directly to a large multi-component architecture.

6. Limitations

A few limits remain. Most papers are application-specific, so their exact numbers should not be transferred directly to my local Docker environment. Several studies also compare MongoDB with other databases rather than isolating Docker networking, storage-volume configuration, or single-host orchestration overhead as standalone variables. Finally, the strongest consistency and geo-distribution results come from cloud multi-region experiments, which I may not be able to reproduce faithfully on one local machine.

7. Conclusion

The main insight from this research phase is that MongoDB is a strong fit for distributed systems when the workload is document-oriented, semi-structured, replication-aware, and ingestion-heavy, but its performance is highly sensitive to schema design, indexing, consistency configuration, and topology. The papers I reviewed show that MongoDB can scale effectively in sharded or containerized environments and integrates naturally with modern cloud-native stacks, yet they also show that stronger consistency, complex analytical queries, and multi-shard coordination can impose substantial costs.

For my own project, this means the next phase should focus on disciplined benchmarking rather than adding complexity too early. I now have a better idea on what I should do next: benchmark the current three-node Docker replica set first, vary workload and consistency settings systematically, measure failover and indexing effects, and only then decide whether expanding toward sharding or a larger microservice architecture is justified.

8. Bibliography:

1. Makris, A., Tserpes, K., Spiliopoulos, G., Zissis, D., and Anagnostopoulos, D. *MongoDB Vs PostgreSQL: A comparative study on performance aspects*. **Geoinformatica**, 25, 243–268 (2021). Access link: <https://doi.org/10.1007/s10707-020-00407-w>
2. Gkamas, T., Karaiskos, V., and Kontogiannis, S. *Performance Evaluation of Distributed Database Strategies Using Docker as a Service for Industrial IoT Data: Application to Industry 4.0*. **Information**, 13(4), 190 (2022). Access link: <https://doi.org/10.3390/info13040190>
3. Mehmood, N. Q., Culmone, R., and Mostarda, L. *Modeling temporal aspects of sensor data for MongoDB NoSQL database*. **Journal of Big Data**, 4, 8 (2017). Access link: <https://doi.org/10.1186/s40537-017-0068-5>
4. Jurado Pérez, L., and Salvachúa, J. *An Approach to Build e-Health IoT Reactive Multi-Services Based on Technologies around Cloud Computing for Elderly Care in Smart City Homes*. **Applied Sciences**, 11(11), 5172 (2021). Access link: <https://doi.org/10.3390/app11115172>
5. Ferreira, S., Mendonça, J., Nogueira, B., Tiengo, W., and Andrade, E. *Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications*.

Journal of Cloud Computing, 13, 158 (2024). Access link:
<https://doi.org/10.1186/s13677-024-00716-7>

6. Pacella, M., Pio, G., and coauthors. *A Scalable Framework for Sensor Data Ingestion and Real-Time Processing in Cloud Manufacturing*. **Algorithms**, 18(1), 22 (2025). Access link: <https://www.mdpi.com/1999-4893/18/1/22>
7. MongoDB Documentation. *Replication*. Access link:
<https://www.mongodb.com/docs/manual/replication/>
8. MongoDB Documentation. *Sharding*. Access link:
<https://www.mongodb.com/docs/manual/sharding/>
9. MongoDB Documentation. *Sharded Cluster Components*. Access link:
<https://www.mongodb.com/docs/manual/core/sharded-cluster-components/>
10. MongoDB Documentation. *Read Concern “linearizable”*. Access link:
<https://www.mongodb.com/docs/manual/reference/read-concern-linearizable/>
11. MongoDB Documentation. *Transactions*. Access link:
<https://www.mongodb.com/docs/manual/core/transactions/>
12. Docker Documentation. *Networking in Compose*. Access link:
<https://docs.docker.com/compose/how-tos/networking/>